CC1101Radio User Manual

ESP-IDF edition. Native `spi_master` driver for the TI CC1101 sub-GHz transceiver.

Updated to match CC1101Radio.cpp and CC1101Radio.h
This revision replaces the former Arduino-centric setup, constructor, SPI transaction, multitasking, delay, and packaging guidance with the behavior implemented by the attached ESP-IDF source.

1. Introduction

CC1101Radio is a C++ driver for the Texas Instruments CC1101 low-power sub-GHz transceiver. This edition is implemented as an ESP-IDF component and uses the native `driver/spi_master.h` and `driver/gpio.h` APIs. The radio logic remains focused on packet-mode operation with polled transmit and receive handling.

- **Supported bands:** 300-348 MHz, 387-464 MHz, and 779-928 MHz.
- **Modulations:** 2-FSK, GFSK, ASK/OOK, 4-FSK, and MSK.
- **Packet modes:** variable, fixed, and infinite-length streaming.
- **Packet features:** CRC, whitening, Manchester, convolutional FEC, hardware address filtering, RSSI/LQI reporting, and frequency-error estimation.
- **Multiple radios:** create one object per CC1101, give each its own CS pin, and attach each device to an already initialized SPI bus.
- **Thread coordination:** each public SPI operation acquires the ESP-IDF SPI bus through an internal RAII BusLock.
- **Clock assumption:** all calculations assume the standard 26 MHz CC1101 crystal.

2. Hardware and wiring

2.1 Connections

Signal	Connect to	Notes
SCK	ESP SPI clock GPIO	SPI mode 0
MOSI / SI	ESP SPI MOSI GPIO	Configured when the bus is initialized
MISO / SO	ESP SPI MISO GPIO	Pass the same GPIO as misoPin for chip-ready polling
CSn	Any output-capable GPIO	Driven manually by the class; hardware CS is disabled
VCC / GND	3.3 V supply	Do not apply 5 V
GDO0 / GDO2	Optional	The driver does not require interrupts; GDO0 defaults to sync/end-of-packet signaling

2.2 Chip-ready handling

After CS goes low, the CC1101 drives SO/MISO low when its crystal is ready. If `misoPin` is nonnegative, the driver polls that GPIO before transfers, with bounded waits. If it is -1, reset and wake paths use a fixed microsecond delay fallback.

2.3 SPI clock

The constructor defaults to 4 MHz. Requested rates above 6.5 MHz are clamped to 6.5 MHz, matching the burst-access limit used by the implementation. Transfers use SPI mode 0. The class registers the device with `spics_io_num = -1` because CS is controlled as a normal GPIO.

3. Component setup

3.1 Files and dependencies

- Add `CC1101Radio.h` and `CC1101Radio.cpp` to an ESP-IDF component.
- The code depends on the ESP-IDF GPIO, SPI master, timer, ROM delay, and FreeRTOS APIs.
- Initialize the SPI bus before calling `begin()`. The class adds its device to that existing bus.

3.2 Minimal CMakeLists.txt

```
idf_component_register(
    SRCS "CC1101Radio.cpp"
    INCLUDE_DIRS "."
    REQUIRES driver esp_timer freertos
)
```

3.3 Bus initialization and quick start

```
#include "CC1101Radio.h"
#include "driver/spi_master.h"

static constexpr int PIN_SCK = 18;
static constexpr int PIN_MISO = 19;
static constexpr int PIN_MOSI = 23;
static constexpr int PIN_CS = 5;

extern "C" void app_main(void) {
    spi_bus_config_t bus = {};
    bus.sclk_io_num = PIN_SCK;
    bus.miso_io_num = PIN_MISO;
    bus.mosi_io_num = PIN_MOSI;
    bus.quadwp_io_num = -1;
    bus.quadhd_io_num = -1;
    bus.max_transfer_sz = 66;

    ESP_ERROR_CHECK(spi_bus_initialize(SPI2_HOST, &bus, SPI_DMA_DISABLED));

    CC1101Radio radio(SPI2_HOST, PIN_CS, PIN_MISO);
    int16_t rc = radio.begin();
    if (rc != CC1101Radio::ERR_NONE) return;

    radio.startReceive();
    for (;;) {
        uint8_t buf[CC1101Radio::MAX_PAYLOAD];
        CC1101Radio::PacketInfo info;
        int16_t n = radio.readPacket(buf, sizeof(buf), &info);
        if (n > 0) { /* consume buf[0..n-1] */ }
        vTaskDelay(pdMS_TO_TICKS(5));
    }
}
```

4. Construction and initialization

4.1 Constructor

```
CC1101Radio(spi_host_device_t host, int csPin, int misoPin = -1,
            uint32_t spiHz = 4000000);
```

Argument	Meaning
host	SPI host already passed to spi_bus_initialize(), for example SPI2_HOST
csPin	GPIO driven by the class as active-low chip select
misoPin	MISO GPIO used for chip-ready polling; -1 uses fixed-delay fallback
spiHz	Requested SPI clock, clamped to 6.5 MHz

The destructor removes the SPI device from the bus if it was attached. Copy construction and assignment are disabled.

4.2 begin()

```
int16_t begin(float freqMHz = 433.92f, float bitrateKbps = 4.8f,
              float devKHz = 47.6f, float rxBwKHz = 135.0f,
              Modulation mod = MOD_GFSK);
```

- Calls the private attach step. If `spi_bus_add_device()` fails, returns `ERR_SPI` (-9).
- Performs the CC1101 manual reset sequence.
- Rejects VERSION values 0x00 and 0xFF as no chip found.
- Loads SmartRF-derived defaults and applies modulation, baud rate, deviation when applicable, bandwidth, frequency, and 0 dBm output power.
- Leaves the chip in IDLE on success. Call `startReceive()` to listen.

4.3 Reset

`reset()` performs the manual CS/SRES sequence. It resets hardware registers but does not reapply the complete configuration. Call `begin()` when a full reinitialization is required.

5. Configuration reference

5.1 RF and modem configuration

Function	Accepted behavior
setFrequency(mhz)	300-348, 387-464, or 779-928 MHz; updates the band-dependent PA table when appropriate
setBitrate(kbps)	2-FSK 0.6-500; GFSK/OOK 0.6-250; 4-FSK 0.6-300; MSK 26-500 kBd
setDeviation(kHz)	1.5-381 kHz; intended for 2-FSK, GFSK, and 4-FSK
setRxBandwidth(kHz)	58-812 kHz; chooses the closest hardware setting and updates analog test values
setModulation(m)	2-FSK, GFSK, ASK/OOK, 4-FSK, or MSK
setChannelSpacing(kHz)	25-406 kHz, nearest achievable value
setChannel(channel)	Writes CHANNR after entering IDLE

Configuration setters enter IDLE before modifying modem or packet fields. If reception should continue, call `startReceive()` afterward.

5.2 ASK/OOK and MSK helpers

- `setOokAgc(magnTarget, agcCtrl0)` is valid only in ASK/OOK. magnTarget must be 0-7.
- `setMskPhaseTransition(m)` is valid only in MSK and accepts 0-7.
- Switching into ASK/OOK disables FSK frequency-offset compensation and applies OOK-specific AGC/FREND1 values. Leaving OOK restores saved FSK values.
- Switching to MSK or 4-FSK disables Manchester.

5.3 Packet configuration

Function	Purpose
setSyncWord(hi, lo)	Sets the 16-bit sync word
setSyncMode(mode)	Selects sync acceptance and optional carrier-sense requirement
setPreambleBytes(n)	Snaps to 2, 3, 4, 6, 8, 12, 16, or 24 bytes
setPreambleQuality(pqt)	Sets PQT, clamped to 0-7
setCrc(enabled)	Enables or disables packet CRC
setCrcAutoflush(enabled)	Hardware flush on bad CRC
setWhitening(enabled)	Enables data whitening
setManchester(enabled)	Rejected with MSK, 4-FSK, or active FEC
setNodeAddress(addr)	Programs ADDR and updates driver state
setAddressFilter(mode)	Off, node only, node + 0x00, or node + 0x00/0xFF

6. Packet length modes

Mode	Receive limit	Transmit behavior
Variable	1-61 bytes	Length byte precedes payload; 1-61 total packet bytes
Fixed	1-62 bytes	No length byte; transmit may retarget PKTLEN for a 1-64 byte call
Infinite	Streaming	Packet transmit and readPacket are unavailable; use readStream

- `setVariableLength(maxLen)` rejects 0 or values above 61 and disables FEC.
- `setFixedLength(n)` accepts 1-62 bytes for receive.
- `setInfiniteLength()` disables FEC. CRC and address filtering do not apply to the stream.
- `setFec(true)` requires fixed-length mode and disables Manchester.

6.1 Addressed packets

`transmitTo(dest, data, len)` inserts the destination byte before the payload. With receive address filtering enabled, `readPacket()` returns that address in `buf[0]`; `PacketInfo` reports whether it matched a configured broadcast address.

7. Transmit and receive

7.1 Transmit

```
int16_t transmit(const uint8_t* data, uint8_t len, uint32_t timeoutMs = 0);
int16_t transmitTo(uint8_t dest, const uint8_t* data, uint8_t len,
                   uint32_t timeoutMs = 0);
```

The driver enters IDLE, flushes TX, writes the complete frame under one bus lock, and strobes STX. With timeout 0, it estimates airtime from preamble, sync, framing, CRC, Manchester/FEC, 4-FSK, and the programmed baud rate, then adds margin. The chip ends in IDLE.

7.2 Receive

```
int16_t readPacket(uint8_t* buf, uint8_t maxLen, PacketInfo* info = nullptr);
int16_t readStream(uint8_t* buf, uint8_t maxLen);
```

- `readPacket()` returns 0 while no complete packet is ready, a positive payload length on success, or a negative error.
- RX overflow, invalid lengths, and undersized destination buffers cause reception to be restarted.
- Packet status bytes provide RSSI, LQI, and CRC-OK. Bad-CRC payload data is still copied before `ERR_CRC` is returned.
- `readStream()` is valid only in infinite mode and deliberately leaves one byte in the FIFO while reception continues.

7.3 PacketInfo

Field	Meaning
rssi	Packet RSSI in dBm
lqi	7-bit link quality value
crcOk	CRC status bit
hasAddress	True when receive address filtering is active
address	Destination byte from buf[0]
broadcast	True when destination matched a configured broadcast address

8. State, power, and diagnostics

8.1 State control

Function	Behavior
idle()	Wakes if necessary, strobes SIDLE, waits for MARC_IDLE
sleep()	Saves PATABLE and test registers, then enters SPWD
wake()	Starts the crystal through CS, restores registers lost in sleep, leaves the chip in IDLE

Function	Behavior
calibrate()	Runs SCAL and waits for IDLE
startReceive()	Enters IDLE, flushes RX FIFO, then strobes SRX

8.2 Output power

Nominal dBm steps are -30, -20, -15, -10, 0, 5, 7, and 10. `setOutputPowerDbm()` selects the nearest step. `setPaRaw()` bypasses table selection. `setInductorType()` selects multilayer or wire-wound recommendations for 868/915 MHz.

8.3 Diagnostics

- `getRSSI()` returns live RSSI using the driver conversion.
- `packetStatus()` and `carrierSense()` expose PKTSTATUS.
- `frequencyErrorKHz()` converts signed FREQEST to kHz.
- `partNumber()`, `version()`, and `marcState()` expose chip identity and state.
- Status registers that may update asynchronously are read repeatedly until two reads agree, with a bounded retry count.

9. SPI sharing and concurrency

The ESP-IDF edition differs materially from the old Arduino manual. Public register, burst, status, and strobe functions acquire the SPI bus with `spi_device_acquire_bus(..., portMAX_DELAY)` and release it through a private RAII guard. Multi-step sequences that must remain atomic use one lock and call private unlocked primitives.

- Do not call a public SPI method while the internal BusLock is already held. The implementation uses unlocked helpers for nested operations.
- Each CC1101 has a separate SPI device handle and software-controlled CS GPIO.
- Transfers are split into chunks of at most 64 bytes, avoiding a dependency on DMA-capable caller buffers for the radio FIFO-sized operations.
- Burst helper lengths are clamped to 64 bytes.
- Use normal application-level synchronization if multiple tasks also mutate higher-level configuration/state on the same object. The bus lock serializes SPI traffic but does not make a compound application workflow transactional.

9.1 Timing behavior

The driver maps microsecond timing to `esp_timer_get_time()` and `esp_rom_delay_us()`. Delays shorter than 10 ms busy-wait; delays of 10 ms or more use `vTaskDelay(pdMS_TO_TICKS(ms))`. Transmit completion is still polled.

10. Error codes

Constant	Value	Meaning
ERR_NONE	0	Success
ERR_CHIP_NOT_FOUND	-1	VERSION read as 0x00 or 0xFF
ERR_BAD_ARG	-2	Unsupported argument or operation in current mode

Constant	Value	Meaning
ERR_TIMEOUT	-3	Transmit did not complete in time
ERR_OVERFLOW	-4	RX FIFO overflow
ERR_CRC	-5	Packet CRC failed
ERR_BUF_SMALL	-6	Destination buffer is shorter than the packet
ERR_LENGTH	-7	Invalid TX or RX packet length
ERR_TX_UNDERFLOW	-8	TX FIFO underflow
ERR_SPI	-9	Could not attach the device to the initialized SPI bus

11. Raw register access

```
uint8_t readReg(uint8_t reg);
void writeReg(uint8_t reg, uint8_t value);
uint8_t readStatus(uint8_t reg);
uint8_t strobe(uint8_t cmd);
void readBurst(uint8_t reg, uint8_t* buf, uint8_t len);
void writeBurst(uint8_t reg, const uint8_t* buf, uint8_t len);
```

The complete configuration-register, status-register, command-strobe, and MARCSTATE maps are defined in `CC1101Reg`. Direct writes are not mirrored into all driver shadow fields, so changing registers managed by high-level methods can make the software state disagree with hardware. Enter IDLE before changing modem or packet registers. Use `sleep()` and `wake()` rather than issuing SPWD directly.

12. Migration notes from the Arduino edition

Topic	Old Arduino guidance	Current ESP-IDF implementation
Constructor	CC1101Radio(csPin, SPI, misoPin, spiHz)	CC1101Radio(host, csPin, misoPin, spiHz)
Bus setup	SPI.begin(...)	spi_bus_initialize(...) before radio.begin()
Device attach	Implicit pin setup only	begin() calls spi_bus_add_device(); may return ERR_SPI
Bus locking	beginTransaction/endTransaction; no object locking	spi_device_acquire_bus/release_bus through BusLock
CS control	digitalWrite	gpio_set_level
MISO ready	digitalRead	gpio_get_level
Timing	Arduino micros/millis/delay	esp_timer, ROM microsecond delay, FreeRTOS delay
Destructor	None	Removes attached SPI device
Burst access	Caller warned to keep length <=64	Driver clamps to 64 bytes
Packaging	Sketch/library files	ESP-IDF component

12.1 Important corrected statements

- The current source is not an Arduino implementation and does not include `Arduino.h` or `SPI.h`.

- The current implementation does have an attach failure path and defines `ERR_SPI = -9`.
- The current implementation internally serializes SPI access with BusLock; the former statement that the object has no locking is no longer accurate.
- The current quick-start constructor and all examples must pass an ESP-IDF SPI host, not an Arduino SPIClass object.

13. Troubleshooting

Symptom	Checks
ERR_SPI (-9)	Confirm spi_bus_initialize succeeded, the selected host is correct, and the bus can accept another device.
ERR_CHIP_NOT_FOUND (-1)	Check 3.3 V power, SCK/MOSI/MISO/CS wiring, host selection, and the misoPin value.
No receive	Verify both ends match frequency, modulation, baud, deviation, sync, length mode, CRC, whitening, Manchester, FEC, and addressing. Call startReceive after configuration or transmit.
ERR_OVERFLOW	Poll readPacket/readStream more frequently.
ERR_LENGTH	Check length mode and over-the-air framing on both ends.
ERR_TIMEOUT	Check power integrity and RF configuration; provide a larger explicit timeout for unusually slow packets if required.
Wrong RF frequency or rate	Verify the module uses a 26 MHz crystal.
Settings lost after reset	Use begin() for a full reset plus reconfiguration.

End of updated manual. API declarations and behavioral descriptions were aligned with the attached header and implementation.